

Judge File Ranking Prompt

```
SYSTEM: You are judging an attempt to reproduce a research paper. You will be shown a paper, a resolution criteria, and a set of files.

The resolution criteria is a specific aspect of the paper that you are marking. The set of files is the submission that someone has made, that is attempting to reproduce the paper.

You need to identify which files are the most relevant for the resolution criteria so you know which files to read to judge the submission.
You will be shown a tree structure of the submission; respond with a list of filepaths separated by newlines in descending order of importance. Each line should be a full filepath.

List all the files that are shown to you, do not leave any out. Do not return anything else.

--

USER: The paper is below:
<paper>
{self.paper_md}
</paper>

--

USER: If included with the paper, you will now be shown an addendum which provides clarification for the paper and how to evaluate its reproduction:
<addendum>
{self.joined_addendum}
</addendum>

--

USER: Here is the resolution criteria that you are grading:
<resolution_criteria>
{task.requirements}
</resolution_criteria>

--

USER: Here are the files in the submission attempt:

Directory structure:
{tree_structure}

Flat file list:
{all_files_str}

Now return a list of the {str(max_files) + ' ' if max_files else ''}most relevant files in order of relevance (descending) to the resolution criteria, to be provided for your inspection. Your response must contain each filename separated by newlines, with each file containing the full path. Use the exact paths from the flat file list. Do not write anything else.
```

Figure 7. Judge file ranking prompt. Variables like {tree_structure} are replaced with their value at runtime.

Judge Prompt. Part 1 of 2

```

SYSTEM: You are judging a candidate's attempt to reproduce a research paper.

You will first be given the research paper, which you should read carefully and refer to as the ground truth for what
constitutes a correct implementation and reproduction.

Following that, you will be presented with the candidate's submission: a set of files that attempts to reproduce the paper. The
candidate has been asked to include a 'reproduce.sh' script that, when executed, should run the full reproduction process
and generate all results in the paper. If the script exists, we have already run it and included any output logs in the
file 'reproduce.log'. Any other files generated by the script may also be present, such as results, plots, or artifacts.
If either 'reproduce.sh' or 'reproduce.log' is missing, you should consider any criteria relying on it to have failed.

To judge the submission, we have prepared a full rubric describing fine-grained criteria for different aspects of the paper.
A given criterion checks exactly ONE of the following aspects of the submission:
1. Code Development - Does the submission's source code contain a correct implementation of this?
2. Execution - Does running the reproduce.sh script lead to this being successfully executed?
3. Result Match - Does the outcome of the reproduction agree with these results?
A given criterion asks:
Does the submission's source code contain a correct implementation of this?

Your task is to check the submission and its outputs for ONE specific criterion from this rubric.

--

USER: The paper is below:
{paper_markdown}

--

USER: If included with the paper, you will now be shown an addendum which provides clarification for the paper and how to
evaluate its reproduction:
<addendum>
{addendum + judge_addendum}
</addendum>

--

USER: Here are the most relevant files included in the submission attempt, concatenated:
USER: Here are the most relevant docs and the files touched (i.e. modified or created) during the reproduce.sh execution,
concatenated:
<files>
{files}
</files>

--

USER: Here is the 'reproduce.sh' provided in the submission, if any:
<reproduce.sh>
{reproduce.sh}
</reproduce.sh>

--

USER: Here is the 'reproduce.log' provided in the submission, if any
<reproduce.log>
{reproduce.log}
</reproduce.log>

```

Figure 8. Judge Prompt. Part 1 of 2. Variables like {paper_markdown} are replaced with their value at runtime. Text in red is omitted from the prompt for PaperBench-CodeDev. Text in blue is exclusively shown for PaperBench-CodeDev. Text in green is shown exclusively for Code Development and Execution leaf nodes while text in purple is shown exclusively for textttResult Match nodes.

Judge Prompt. Part 2 of 2

```
--

USER: The full grading rubric for the paper hierarchically decomposes various aspects of the paper. To provide more context
      for the criterion you are currently grading, here are some of the preceding criteria (DO NOT grade this, this is just for
      context):
<preceding_criteria>
{preceding_criteria}
</preceding_criteria>

Now, here is the criterion that you are grading:
<criterion>
{criterion}
</criterion>

The criterion you are grading is of type {task_category} - {task_category_question}

--

USER: You will now evaluate the submission above for the given criterion.

Please present your answer in 3 parts as follows:

# Expectations
Read the paper carefully to identify the parts relevant to your criterion. Describe what you expect correct resolution to look
like, and what you will specifically look for in the submission.

# Reality
Explore the files provided for the submission along with the output logs to identify the parts that are relevant to the
resolution criteria. When you have identified the relevant parts, comment on how they compare to your expectations above,
and whether the attempt meets the resolution criteria. Be sure to describe only what has been given to you in the
submission, and be explicit about which files you are referring to when you discuss them.

# Score
Reason about whether the criteria has been met. Give a score of either 0 or 1 depending on whether the criteria has been met,
and explain why."

Other notes:
- You must always provide a score. If you have any uncertainties, make them clear in your discussion.
- All the files and logs from the submission attempt have been provided to you. If anything appears to be missing, assume that
the attempt failed to produce it (treat it as a failure).
- Be strict and thorough in grading your resolution criteria, but do not check for things that are outside of your scope.
```

Figure 9. Judge Prompt. Part 2 of 2. {criterion} corresponds to the requirements of the leaf node currently being graded. {preceding_criteria} are the requirements of preceding nodes (siblings and direct ancestors). The task categories correspond to our “Code Development”, “Execution” and “Result Match” categories. The corresponding {task_category_question} are “Does the agent’s source code contain a correct implementation of this?”, “Does running the reproduce.sh script lead to this being successfully executed?”, and “Does the outcome of the reproduction agree with these results?”.

BasicAgent System Prompt

You are a helpful agent attempting to solve a task. You have several functions available to help with solving the task. Each message may may perform one function call. You will see the result of the function right after sending the message. If you need to perform multiple actions, you can always send more messages with subsequent function calls. Do some reasoning before your actions, describing what function calls you are going to use and how they fit into your plan.

Use as much of the available time as possible to refine and optimize your solution: Your goal is to achieve the best possible score, not just to submit your first working solution. Keep working until the time limit expires.

YOU MUST use the bash and python tools provided provided to create your replication. ACTUALLY REPLICATE the paper YOURSELF, do not give me instructions how I can do so.

Only execute a single command using each tool at once.

This is a multi-step task. DO THINGS BIT BY BIT by slowly building up your solution with multiple tool calls, don't try to do everything in one go. Just reason about what is the best next step and execute it. Then take the next step after you receive the output of the previous step.

Do not stop until you have replicated all results from the paper. Before calling end_task, YOU MUST have an entire codebase that can reproduce all metrics, figures, tables, and findings from the paper

Ensure that running reproduce.sh will reproduce all metrics, figures, tables, and findings from the paper! You should check reproduce.sh YOURSELF to ensure that it actually reproduces all results from the paper to help with your development.

Remember, you are tasked with replicating a whole paper, not just a single part of it or a minimal example. The file read tool is PAGINATED, so you will need to CALL IT MULTIPLE TIMES to make sure that you have read all the relevant parts of the paper.

Figure 10. BasicAgent System Prompt

IterativeAgent System Prompt

You are a helpful agent attempting to solve a task.

- You have several functions available to help with solving the task. Each message may perform one function call. You will see the result of the function right after sending the message. If you need to perform multiple actions, you can always send more messages with subsequent function calls.
- Do some reasoning before your actions, describing what function calls you are going to use and how they fit into your plan.
- Use as much of the available time as possible to refine and optimize your solution: Your goal is to achieve the best possible score, not just to submit your first working solution. Keep working until the time limit expires.
- The file read tool is paginated, so you will need to call it multiple times to make sure that you have read all the relevant parts of the paper.
- This is a multi-step task. DO THINGS BIT BY BIT by slowly building up your solution with multiple tool calls, don't try to do everything in one go. Just reason about what is the best next step and execute it. Then take the next step after you receive the output of the previous step.
- You should try and create a clean codebase. When it makes sense you should edit/overwrite files rather than creating new ones.
- When writing code, do not write newlines as the string "\n", instead use the newline character.

Figure 11. IterativeAgent System Prompt

IterativeAgent Continue Message

Now given the previous progress made by the agent, take the next step towards replicating the paper:

- You have a lot of time available, so don't try and do everything in one go. Just reason about what is the best next step and execute it.
- Use the available tools to write and execute the necessary code.
- Remember, you should try prioritize the most important parts of the paper to replicate first.

Figure 12. IterativeAgent Continue Message

Task instructions. Part 1 of 2

You are tasked with reproducing a research paper.

You have been provided with a research paper located at `~/home/paper/`. The directory contains the paper in both PDF and markdown format.

TASK

Your task is to replicate as many of the core contributions of this paper as possible, within the allotted time you have available. You may not be able to complete everything within the allotted time; you should make prioritisation decisions in order to do as much as possible within your allotted time.

Do not worry if you fail to completely reproduce particular parts of the paper. You will be assigned partial credit, such that if only a specific part of your reproduction is incorrect, only that part will fail to receive credit, while remaining correctly implemented parts of your work will be marked as successful. Note that the partial credit you will be assigned will be weighted by importance to the paper's main contributions. Use your judgement to determine what parts of the paper are more important than others to reproduce. Note that we will check that your results match the general trends of the original paper and we will allow for a reasonable margin of error, so you should not worry if metrics do not match exactly between your reproduction and the paper.

Any experiments that are only introduced in the Appendix are not considered core contributions of the paper, and so are out of scope. This is the case even if the experiment is referenced in the main text. HOWEVER, if an experiment is described in the main body of the text, but some of the details used (e.g. details about a prompt used, or a long mathematical expression) have been put in the appendix, then that experiment is considered still in scope. The reason here is that the experiment is described in the main body of the paper.

RESOURCES

You may or may not be provided with a rubric called `'rubric.json'` in the `~/home/paper/` directory to guide your efforts. The `'rubric.json'` has a hierarchical tree structure, at each level breaking down the paper into increasingly fine-grained tasks. Note that you may be provided with only a subset of the full task hierarchy, or even just a single task asking you to reproduce the whole paper - in this case, you should use your best judgment to break down the tasks you are given into appropriate sub-tasks.

There is also an addendum file at `~/home/paper/addendum.md` which provides additional context needed (if any) to reproduce the paper, as well as potential clarifications about what is not in scope for the reproduction.

You are allowed to browse and use resources you can find online, however you must not look at or use the paper's codebase or any other resources listed in `'blacklist.txt'`, as this is considered cheating.

SUBMISSION

The output of your work must be a git repository containing all the necessary components to reproduce the results of the paper. Your repository should be located at `~/home/submission/`, and the committed files should not exceed 1GB in size (i.e. you should only commit source code without any heavy artifacts). We will run `'git clean -fd'` on the submission directory before grading it, so any untracked files will be lost.

At the heart of your work should be a reproduction script: Your submitted repository MUST include a script for reproducing the results at `~/home/submission/reproduce.sh`. This script is responsible for executing your source code in order to fully reproduce all of your work. We will run this script in a fresh Ubuntu 24.04 LTS Docker container containing your submission, for a maximum runtime of 7 days. The container will have access to an NVIDIA A10 GPU, with the NVIDIA container toolkit already installed. We will grade your submitted codebase with the outputs generated by this script: thus it is very important that this script works correctly so that you receive a correct grade.

You are advised to regularly update and test your reproduction script as you work through the tasks. Docker has been installed in your environment, should you wish to use it.

Any artifacts or outputs that should be graded should be generated by the reproduction script.

Finally, please also include a `README.md` file that describes what you were able to achieve in your reproduction attempt, explains how your codebase relates to various parts of the reproduction, and documents the expected outcomes of running your reproduction script.

Figure 13. Task Instructions, continued in Part 2

Task Instructions. Part 2 of 2

```

REPRODUCTION SCRIPT TOY EXAMPLE
-----

**Imagine the following toy paper**:

'''
We count the number of 'r's in the word 'strawberry' using a python script, and find that there are 3 instances of the letter
    'r'
'''

> the paper measures the number of 'r's in the word strawberry (this is an artefact, think of this as a table or figure or
    result), using a basic python script as an implementation (think of this as an algorithm described in a paper)

**Toy Submission**:

submission/count.py

'''python
import argparse, csv

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--word', default="strawberry")
    parser.add_argument('--output', default="output.csv")
    args = parser.parse_args()

    r_count = args.word.lower().count('r')
    with open(args.output, 'w', newline='') as f:
        csv.writer(f).writerows([["word", "r count"], [args.word, r_count]])

    print(f'{args.word} has {r_count} 'r'(s). Saved to '{args.output}'.')

if __name__ == "__main__":
    main()
'''

submission/reproduce.sh

'''bash
apt-get update && apt-get install -y python3

# Run the Python script with the specified arguments
python3 count.py --word strawberry --output output.csv

# Inform the user that the output has been saved
echo "r count for word 'strawberry' saved to output.csv"
'''

**How we would grade this**:
- Implementation: correct, a python script for counting "r"s in "strawberry" was implemented
- Execution: correct, the python script for counting "r"s in "strawberry" was correctly **executed by the reproduce.sh**
  script
- Result: inspecting the output.csv **produced by reproduce.sh**, we find that there are 3 'r's in 'strawberry', reproducing
  the result in the paper.

ADDITIONAL NOTES
-----
- **Compute**: You have access to a ${type_of_processor} with the appropriate drivers installed.
- **Total Runtime**: You have a maximum of ${max_time_in_hours} hours to attempt this task. You may iteratively develop your
  solution within this time frame, but bear in mind that program runtime also counts towards this limit. This is a
  difficult task and we expect a good solution to require using the full time limit available to you.
- **Total Steps**: You have a maximum of ${max_steps} steps to submit your solution. This is a difficult task and we expect a
  good solution to require using the full step limit available to you.
- **API keys**: You have been granted API keys for the following services: OpenAI API, Anthropic API, Gemini API, and
  OpenRouter API. The API keys are available to you at '${workspace_base}/agent.env'. You can assume that these same API
  keys will already be set as environment variables in the reproduction environment when we evaluate your submission, so
  you do not need to set or export them in your submission.
- You have root access in your environment.
- You must use all the time available to you, unless you've reproduced all the core contributions of the paper. Do not stop
  until you've reproduced them.
- Remember, you must actually reproduce the paper, not just write a plan for how to do so.

'''

```

Figure 14. Task Instructions, continued from Part 1